Knowledge Distillation and Deployment of a Lightweight Command-Parsing Language Model on a Local Cloud

Course: Cloud Computing (CS3081) Mini Project | **Student Name**: Bhukya Prabhas | **Roll Number**: 230103031 |
GitHub: https://github.com/bhukyaprabhas/Cloud_computing_assignment

---

1. Executive Summary & Architecture Overview

This project implements an end-to-end, production-grade cloud computing and machine learning pipeline that distills natural language command parsing from a 3.8B parameter teacher (**Phi-3.5-mini-instruct**) into an ultra-lightweight 70M parameter student model (**EleutherAI/pythia-70m**). The resulting model is containerized with CPU-optimized PyTorch (<1.5 GB footprint) and deployed onto a local multi-tier Kubernetes cluster with automated autoscaling, rolling updates, network access controls, and CI/CD quality gates.

2. Task 1: Object Storage and Parallel Teacher Labelling

2.1 Local Kubernetes Environment & SeaweedFS Object Storage

A lightweight, multi-tenant Kubernetes cluster was instantiated using **k3d** with Traefik Ingress enabled, port-mapping HTTP traffic (8080:80) and HTTPS (8443:443) via cluster/k3d.yaml. Inside the cluster, an isolated namespace data hosts **SeaweedFS**, an open-source, S3-compatible distributed object storage system. SeaweedFS manages two distinct buckets: datasets for command corpora and teacher shards, and models for serialized fine-tuned PyTorch artifacts. The 3,000 unlabelled SLURP commands (data/commands_pool.jsonl) were ingested into the datasets bucket.

2.2 Sharding Strategy & Modulo Partitioning

To achieve linear labelling speedup across multiple workers without cross-worker locking or coordination overhead, the 3,000 command dataset was partitioned using a deterministic **modulo sharding strategy**. Each worker runs in the pipeline namespace and reads two environment variables: SHARD_INDEX and SHARD_COUNT. For any line index $i \in \{0, 1, \dots, 2999\}$:

> **Partition Invariant:** Worker k processes command i if and only if $i \equiv k \pmod{K}$, where $K = \text{SHARD_COUNT}$.

For $K=2$, Worker 0 processed all even lines (1,500 commands $\to$ shard-0.jsonl) and Worker 1 processed all odd lines (1,500 commands $\to$ shard-1.jsonl). Because the residue classes modulo K partition the integers into disjoint subsets whose union is the complete set, this guarantees **zero omissions (no gaps)** and **zero overlaps (no repeats)**. Every command ID from C0001 to C3000 appeared in exactly one shard output.

2.3 Data Cleaning Pipeline & Invariant Verification

A robust cleaning pipeline (worker/clean.py) ingested the shard outputs and filtered records against five strict criteria: (a) unparseable JSON, (b) schema key mismatch, (c) intent outside the 8 permitted classes, (d) slot hallucinations not present in the input text, and (e) duplicate commands (retaining the lowest ID). All surviving records were sorted by ID and written to datasets/v1/train.jsonl.

Category / Reason	Count	Description / Criterion
Total Input Commands	3,000	Aggregated outputs from shard-0 and shard-1
(a) Invalid JSON	3	Syntax errors, cut-off strings, or unescaped characters
(b) Incorrect Keys	4	Keys did not match exact set {intent, date, time, place}
(c) Invalid Intent	4	Intent was not one of the 8 canonical classes
(d) Hallucinated Slots	5	Slot text did not appear verbatim in input command
(e) Duplicate Commands	48	Repeated command text; lowest command ID preserved
Cleaned Records Kept	**2,936**	Strictly valid records written to datasets/v1/train.jsonl

Mathematical Invariant Check: $3000 = 3 + 4 + 4 + 5 + 48 + 2936 = 3000$ (Verified exact).
Cryptographic Checksum (datasets/v1/train.sha256):
```
eb73fe6451477fd0f7e498f3589150529d134ee3dddd0b435a45f6b1beb9de6a  train.jsonl
```

3. Task 2: Training the Student (Sequence-Level Distillation)

3.1 Sequence-Level vs. Logit-Based Distillation

Knowledge distillation transfers inductive biases from a high-capacity teacher to a compact student. The two primary paradigms are: **(1) Logit-Based Distillation**: The student minimizes the Kullback-Leibler (KL) divergence over the teacher's full vocabulary output distribution: $\mathcal{L}_{\text{logit}} = \alpha \mathcal{L}_{\text{CE}} + (1-\alpha) T^2 D_{\text{KL}}(\sigma(z_s/T) \parallel \sigma(z_t/T))$, where z_s, z_t are logits and T is temperature. **(2) Sequence-Level Distillation (Kim & Rush, 2016)**: The student trains via standard cross-entropy solely on the teacher's greedily decoded output sequences: $\mathcal{L}_{\text{seq}} = -\sum_{t=1}^N \log P_s(y_t^* \mid y_{1:t-1}^*, x)$, where $y^* = \arg\max_y P_t(y \mid x)$.

Why Logit-Based Distillation is Infeasible in this Project:
1. **Vocabulary Mismatch**: Phi-3.5 uses a 32,064-token SentencePiece/tiktoken vocabulary, whereas Pythia-70M uses a 50,304-token GPT-NeoX byte-level BPE vocabulary. Softmax distributions cannot be directly compared across disparate tokenizations without learning an auxiliary projection matrix.
2. **Black-Box API Constraints**: Phi-3.5 was served via Ollama over HTTP, which outputs generated text tokens. Extracting complete 32,064-dimensional float32 logit tensors for every token step over HTTP would incur catastrophic network transfer latency and memory bloat on laptop hardware.
3. **CPU Compute Feasibility**: Computing KL divergence across 50,000 vocabulary logits on every token step on CPU hardware introduces substantial overhead, whereas sequence-level cross-entropy is extremely fast.

Real Limitation of this Approach:
Sequence-level distillation discards the teacher's *dark knowledge*—the relative probability margins between alternative candidates (e.g., whether the second-best intent was calendar_set or alarm_set). The student treats teacher outputs as hard targets, leading to occasional overconfidence on mislabelled slots and limited generalization on ambiguous utterances.

3.2 Training Implementation & Engineering Optimizations

Training was executed entirely on CPU using PyTorch and Hugging Face Transformers (training/train.py). To resolve critical CPU performance bottlenecks, three engineering solutions were implemented: (1) **Float32 Precision Casting**: Pythia-70M weights are stored in float16. On CPU, AdamW updates on float16 cause gradient underflow and immediate NaN values; casting to float32 ensured numerical stability. (2) **Prompt Loss Masking**: Input prompts were formatted as Command: {text}\nJSON:\n{raw_json}\n. Tokens preceding JSON:\n had labels masked to -100 so gradients backpropagated strictly through target JSON tokens. (3) **Thread Affinity**: Windows OpenMP runtime contention was eliminated by fixing torch.set_num_threads(2), reducing batch step time from 44s to 0.77s.

Hyperparameter	Configured Value	Technical Justification
Base Model	EleutherAI/pythia-70m	70M parameter GPT-NeoX causal architecture
Training Records	1,500 records	First 1,500 records of train.jsonl (Section 6)
Random Seed	42	Fixed in code and config for reproducibility
Learning Rate	1e-4 (AdamW)	Stable convergence without gradient explosion
Batch Size	8	Optimized for CPU cache and vectorization
Epochs	3 (v1) / 4 (v2)	Model v1 converges at epoch 3; v2 adds 1 epoch
Max Seq Length	64 tokens	Max command + JSON length in train set is 60 tokens

3.3 Training Loss Progression & Model Checksums

Training Loss Log (training/loss.csv): Epoch 1: **0.1788** | Epoch 2: **0.0704** | Epoch 3: **0.0491**.
Model v1 SHA-256 Checksum (models/v1/model.sha256):

```
7cab8da176fd5086a13d807eda6bdd4260e067de02855b710448d17a9b430b9f  model.safetensors
28af6644622beebd28a7f2ec7669223e7b4eef8aac30e8fc441bde121ca9d36e  tokenizer.json
```

3.4 Side-by-Side Model Evaluation on Gold Validation Set

All three models were evaluated on the 100 gold validation examples (data/validation.jsonl) using tools/score.py. Results are tabulated side-by-side below:

Evaluation Metric	Untrained Pythia-70M	Phi-3.5 Teacher	Distilled Student (v1)	Rubric Requirement
JSON Validity	0.0%	100.0%	**99.0%**	≥ 95.0% (PASS)
Intent Accuracy	0.0%	75.0%	72.0%	Close to teacher
Date Accuracy	0.0%	86.0%	78.0%	-
Time Accuracy	0.0%	93.0%	91.0%	-
Place Accuracy	0.0%	93.0%	89.0%	-
Full-Frame Accuracy	0.0%	59.0%	**46.0%**	≥ 29.5% (half teacher) (PASS)

Analysis: Distillation transformed an untrained base model (0% syntax compliance) into an accurate parser with **99.0% JSON validity** and **46.0% full-frame accuracy**, comfortably exceeding the 29.5% minimum required threshold (half of teacher's 59.0%).

4. Task 3: The Inference Service in a Container

4.1 Microservice Architecture & API Contract

The inference microservice was implemented in FastAPI (service/app.py) running on port 8000. It adheres strictly to the required contract: (1) POST /parse: Ingests {"text": "..."}, invokes the local model with greedy decoding, verifies compliance against data/schema.json, and returns HTTP 200 with the 4-key JSON. It attaches X-Model-Version and X-Served-By headers. (2) GET /healthz: Liveness probe returning HTTP 200 while process is alive. (3) GET /readyz: Readiness probe returning HTTP 503 during startup and HTTP 200 once weights are fully loaded into memory.

4.2 Startup Model Ingestion & Fault Injection

At startup, the service reads four configuration environment variables: MODEL_URI, MODEL_VERSION, S3_ENDPOINT, and MAX_NEW_TOKENS. If MODEL_URI specifies an S3 path (e.g. s3://models/v1), the service dynamically downloads weights from SeaweedFS using boto3 with credentials supplied via a Kubernetes Secret. In accordance with Section 6, **no weights are baked into the container image**.

Fault Injection Testing: The service supports FAULT_INJECT=1. When enabled, the model output string is intercepted and replaced with not json prior to schema verification. The validator catches the malformed payload and immediately emits HTTP 500 with body {"error": "invalid_model_output"}, proving that malformed JSON is never returned with HTTP 200.

4.3 Container Image Optimization (<2 GB Requirement)

A multi-stage, hardened Dockerfile was constructed based on python:3.11-slim (service/Dockerfile). To avoid the massive overhead of standard PyTorch distributions (>4 GB due to unneeded CUDA/cuDNN libraries), the container pins **CPU-only wheels** from PyTorch's official repository: pip install torch==2.11.0 --index-url https://download.pytorch.org/whl/cpu. Every dependency is strictly pinned in service/requirements.txt. The service runs under an unprivileged user (appuser, UID 1000). The resulting container image footprint is **1.28 GB**, well below the 2.0 GB ceiling.

4.4 Stress Testing under Resource Constraints (1 CPU, 1 GiB RAM)

The container was executed under a strict hardware boundary: --cpus=1.0 --memory=1024m. A benchmark sequence of 50 consecutive validation requests was dispatched. All 50 requests completed successfully with **zero out-of-memory (OOM) kills**.

Stress Test Metric	Observed Value	Operational Threshold
Requests Executed	50 / 50	100% completion required
Failed / OOM Requests	0	0 OOM allowed
Peak Memory Consumption	**618 MiB**	< 1,024 MiB (1 GiB Limit)
Average CPU Utilization	68.4%	< 100% (1 CPU Core)
Median Latency per Request	108 ms	< 30,000 ms timeout
Container Image Size	**1.28 GB**	< 2.00 GB

Peak Memory Verification Evidence:

```
CONTAINER ID  NAME  CPU %  MEM USAGE / LIMIT  MEM %  PIDS
8f4b1c2e9a01  parser  74.2%  618.4MiB / 1GiB  60.39%  8
```

5. Task 4: Kubernetes Deployment, Scaling, and Reliability

5.1 Multi-Tier Kubernetes Architecture

The deployment topology separates data and application planes across dedicated namespaces: (1) data hosts SeaweedFS master, volume, and S3 gateway. (2) app hosts the parser application (k8s/deployment.yaml), internal service (k8s/service.yaml), Traefik Ingress (k8s/ingress.yaml), HorizontalPodAutoscaler (k8s/hpa.yaml), and S3 credentials Secret (k8s/secret.yaml).

Resource Bounds & Probe Lifecycle: Each Pod requests cpu: 250m, memory: 512Mi and is capped at cpu: 1000m, memory: 1Gi. During model download and initialization, /readyz returns HTTP 503. Kubernetes keeps the Pod in unready state, ensuring Traefik routes zero ingress traffic until weights are loaded. Once ready, traffic begins flowing seamlessly.

5.2 Autoscaling (HPA) & Load Balancing

A Horizontal Pod Autoscaler was configured with minReplicas: 1, maxReplicas: 2 targeting **70% average CPU utilization**. Under Locust load (4 concurrent users), Pod CPU climbed to ~78%, triggering HPA to scale out from 1 to 2 replicas within 24 seconds. Following load cessation, the cooldown period smoothly scaled the replica count back to 1. Under 2 ready replicas, the Locust load test routed through Ingress distributed traffic evenly: Replica 1 handled **514 requests (51.4%)** and Replica 2 handled **484 requests (48.4%)**, comfortably satisfying the $\ge 30\%$ requirement.

5.3 Rolling Update & Rollback Verification

While Locust load was running continuously, a rolling update was triggered changing MODEL_VERSION and MODEL_URI from v1 to v2: kubectl set env deployment/parser-deployment MODEL_VERSION=v2 MODEL_URI=s3://models/v2. With maxSurge: 1, maxUnavailable: 0, Kubernetes spun up new v2 Pods and waited for readiness probes before terminating v1 Pods. **Zero requests failed (0.0% failure rate $\le$ 0.5% allowed)**. A rollback via kubectl rollout undo restored v1 without dropped connections.

5.4 Pod Recovery & Cold-Start Profiling

Pod Failure Recovery: During active load, one running Pod was forcefully terminated (kubectl delete pod). Kubernetes immediately scheduled a replacement. Ingress routed existing requests to the remaining healthy replica. Failure rate was **0.1% (1 dropped request $\le$ 2.0% allowed)**.
Cold-Start Profiling: All pods were deleted simultaneously. Pod startup time from creation to Ready state was measured across three consecutive trials:

Run	S3 Download Phase	Weight Loading Phase	Total Cold-Start Time
Trial 1	8.4s	5.7s	14.1s
Trial 2	8.8s	5.7s	14.5s
Trial 3	8.2s	5.7s	13.9s
Median Cold-Start	**8.4s**	**5.7s**	**14.1 seconds**

Local Single-Node Cluster Limitation:
The cluster executes on a single laptop node via k3d/Docker. While it validates Kubernetes controller abstractions, it does not demonstrate physical machine failover, hardware rack fault isolation, or cross-node network partition tolerance.

6. Task 5: Subnetting and Network Access Control

6.1 VLSM Subnet Allocation over 10.20.0.0/16

The address space 10.20.0.0/16 was divided into four subnets using Variable Length Subnet Masking (VLSM), ordered strictly from largest requirement to smallest as mandated by Section 7.5. Usable hosts follow the standard formula: $N_{\text{usable}} = 2^{32-\text{prefix}} - 2$.

Subnet Name	Required	Network Address	Prefix	Usable Host Range	Broadcast	Usable
app	1,000	10.20.0.0	/22	10.20.0.1 – 10.20.3.254	10.20.3.255	1022
public	250	10.20.4.0	/24	10.20.4.1 – 10.20.4.254	10.20.4.255	254
data	100	10.20.5.0	/25	10.20.5.1 – 10.20.5.126	10.20.5.127	126
management	20	10.20.5.128	/27	10.20.5.129 – 10.20.5.158	10.20.5.159	30

6.2 Stateless Network Access Control (NACL) Implementation

The NACL for the public subnet (where the load balancer resides) was constructed in network/nacl.json. It enforces strict boundary filtering: (1) Blocks the abusive CIDR 198.51.100.0/24 completely on all ports. (2) Allows incoming web traffic on TCP ports 80 and 443 from anywhere. (3) Restricts SSH management access (TCP 22) exclusively to the management subnet (10.20.5.128/27). (4) Allows return replies from the app backend subnet (10.20.0.0/22) from port 8000 to ephemeral ports 1024–65535. (5) Outbound: allows forwarding to app on TCP 8000 and return replies to clients on ephemeral ports. Default deny catch-all.

Rule #	Direction	Action	Source CIDR	Dest CIDR	Ports	Purpose / Description
50	Inbound	**DENY**	198.51.100.0/24	0.0.0.0/0	Any	Block abusive range before allow rules
100	Inbound	ALLOW	0.0.0.0/0	0.0.0.0/0	80, 443	Allow incoming HTTP and HTTPS
110	Inbound	ALLOW	10.20.5.128/27	0.0.0.0/0	22	Allow SSH from management subnet only
120	Inbound	ALLOW	10.20.0.0/22	0.0.0.0/0	1024-65535	Allow response replies from app backend
100	Outbound	ALLOW	0.0.0.0/0	10.20.0.0/22	8000	Forward proxy requests to app subnet
110	Outbound	ALLOW	0.0.0.0/0	0.0.0.0/0	1024-65535	Return web replies to client ephemeral ports

6.3 Technical Analysis: Stateless Routing & Rule Order Precedence

1. Why Stateless NACLs Require Explicit Return Rules:
Unlike stateful firewalls (e.g. AWS Security Groups) which maintain connection state tables tracking the TCP three-way handshake, a **stateless NACL has zero connection memory**. Every packet is evaluated independently against the rule list. When an external client sends a SYN packet to port 80, the inbound rule allows it; however, the outgoing SYN-ACK reply is a distinct packet heading to the client's ephemeral port (1024–65535). Without an explicit outbound rule permitting traffic to ephemeral ports, the reply packet is dropped by the default deny rule.
2. Why Rule Order Matters for the Abusive Range (198.51.100.0/24):
NACL engines evaluate rules in strict ascending numerical order (first match wins). Rule 50 is configured as DENY 198.51.100.0/24 and Rule 100 is ALLOW 0.0.0.0/0 on port 80. When an abusive client from 198.51.100.15 attempts an HTTP connection, Rule 50 matches first, dropping the packet immediately. If the deny rule were placed at Rule 150, Rule 100 would match first, permitting abusive traffic to access web ports.

Verification Simulation (tools/nacl_sim.py):

```
python tools/nacl_sim.py network/nacl.json network/packets_sample.jsonl --check
Packet 1: ALLOW (expected ALLOW)  |  Packet 2: DENY (expected DENY)  |  Packet 3: ALLOW (expected ALLOW)
Packet 4: DENY (expected DENY)  |  Packet 5: ALLOW (expected ALLOW)  |  Packet 6: DENY (expected DENY)
Result: 6/6 correct (100.0% PASS)
```

7. Task 6: Reproducibility, Testing, and CI/CD

7.1 Unified Makefile Interface

Full lifecycle operations are codified into a declarative Makefile: (1) make up: Builds and deploys the entire stack from a fresh clone in under 3 commands. (2) make down: Destroys the cluster cleanly. (3) make train: Trains the student model reading training/config.yaml. (4) make train SUBSET=50: Runs a rapid 50-example verification training. (5) make test: Runs automated unit tests.

7.2 Automated CI/CD Workflow (.github/workflows/ci.yml)

A robust GitHub Actions pipeline was configured with five distinct stages in the required sequence:

Stage	Tool / Command	Stage Purpose & Verification Criterion
1. Linting	flake8	Enforces PEP8 compliance and detects syntax defects in service, training, worker
2. Unit Tests	pytest tests/test_service.py	Validates 4 test suites: healthz/readyz probes, /parse schema, fault injection, error handling
3. Secret Scan	gitleaks/gitleaks-action	Scans git history for credentials, tokens, or private keys; reports zero leaks
4. Model Download	GitHub Release Asset	Downloads serialized models/v1 release asset and evaluates on validation.jsonl
5. Quality Gate	score.py --min-validity 95	Enforces strict automated deployment quality gate; aborts if validity < 95.0%

7.3 Quality Gate Enforcement & Security Verification

Quality Gate Failure Demonstration: To verify the quality gate prevents defective models from reaching production, the pipeline was triggered with the untrained Pythia-70M baseline. The evaluation produced json_validity: 0.0%. The gate executed score.py --min-validity 95, cleanly exiting with status 1 and halting deployment. (Failed pipeline run: https://github.com/bhukyaprabhas/Cloud_computing_assignment/actions/runs/1089241).
Secret Scanning Audit: Gitleaks scanned all commits and verified zero credentials or keys committed.

8. Task 7: Performance, Cost, and Review

8.1 Load Test Measurements (Locust Benchmark)

The deployed service was load-tested using Locust (tools/locustfile.py) running 4 concurrent users for 2 minutes against the Ingress endpoint (http://localhost:8080/parse). Summary metrics:

Performance Metric	Measured Value	Assessment Threshold	Operational Status
Total Requests Dispatched	1,000 requests	120 second duration	Target Met
System Throughput	8.33 req / second	Sustained 4-user concurrency	Optimal
Median Latency (p50)	110 ms	< 1,000 ms	Excellent
95th Percentile Latency (p95)	**245 ms**	≤ **5,000 ms**	**PASS**
Failure Rate	**0.2% (2 failed)**	≤ **1.0%**	**PASS**
Pod CPU Utilization	68.4% (Replica 1) / 71.2% (Replica 2)	Limit: 1000m (1 core)	Within bounds
Pod Memory Consumption	618 MiB / 624 MiB	Limit: 1,024 MiB (1 GiB)	No OOM
Cold-Start Readiness Time	14.1 seconds (median)	S3 pull + model load	Acceptable

8.2 Cloud Cost Modeling (Step-by-Step Derivation)

Using hypothetical on-demand rates from data/rate_card.json (■3.50 / vCPU-hour, ■0.45 / GiB-hour), cost per 1,000 requests was computed step-by-step:

Step 1: Hourly Cost per Replica:
$\text{Cost}_{\text{replica}} = (\text{CPU Limit} \times \text{Rate}_{\text{cpu}}) + (\text{Mem Limit} \times \text{Rate}_{\text{mem}}) = (1.0 \times ■3.50) + (1.0 \times ■0.45) = ■3.95\text{ / hour}$.
Step 2: Hourly Cost for Active Cluster (2 Running Replicas):
$\text{Cost}_{\text{total}} = 2 \times ■3.95 = ■7.90\text{ / hour}$.
Step 3: Hourly Request Throughput:
$\text{Requests / Hour} = 8.333\text{ req/s} \times 3,600\text{ s/hr} = 30,000\text{ requests / hour}$.
Step 4: Cost per 1,000 Requests:
$\text{Cost}_{1000} = \left(\frac{■7.90}{30,000}\right) \times 1,000 = \mathbf{■0.263\text{ INR}}$ (approx ■0.26 per 1,000 requests).

8.3 Service Level Objective (SLO) & Error Budget

- **SLO Target**: **99.0% of requests must succeed** (HTTP 200 within 30s) over the measurement window.
- **Measurement Window**: One standard 2-minute Locust load test (1,000 total requests).
- **Total Error Budget**: $(1.00 - 0.990) \times 1,000 = \mathbf{10\text{ allowed failed requests}}$.
- **Error Budget Consumed**: 2 actual failed requests occurred ($2 / 10 = \mathbf{20.0\%\text{ consumed}}$).
- **Remaining Error Budget**: 8 requests (**80.0% remaining**). The service comfortably met its SLO.

8.4 Comprehensive Architectural Review Table

The table below details measured evidence, operational limitations identified, and proposed architectural improvements across all six system components:

Component / Concern	Evidence (Measured)	Limitation Identified	Proposed Improvement
Data & Storage	2,936 kept of 3,000; 64 bad records caught	Heuristic regex cleaning misses nuanced slot boundary semantics.	Integrate an active learning validation loop with teacher confidence score filtering.
Model Quality	99.0% JSON validity; 46.0% full-frame	Student hallucinates slots on rare or complex place names.	Employ hybrid sequence-level + LoRA adapter fine-tuning on attention projections.
Latency & Throughput	p50: 110ms; p95: 245ms; 8.33 req/s	Autoregressive token-by-token CPU generation latency scales linearly with length.	Quantize model to INT8 via ONNX Runtime / OpenVINO with static KV-caching.
Resource Utilisation	618 MiB peak RAM; 68.4% CPU per Pod	Cold memory overhead is dominated by PyTorch framework runtime.	Export model to TorchScript / llama.cpp C++ inference runtime to reduce footprint to <150 MiB.
Reliability & Recovery	14.1s cold start; 0.1% pod drop failure	Cold start incurs latency delay while downloading weights from S3 at startup.	Implement persistent ReadWriteMany volume caching or warm standby replica pools.
Security & Cost	■0.26 / 1k requests; 0 gitleaks detected	Static S3 credentials in Kubernetes Secret lack automated IAM rotation.	Adopt HashiCorp Vault / SPIFFE workload identity with short-lived tokens.

9. Conclusion

This project successfully demonstrated that an ultra-compact 70M parameter language model fine-tuned via sequence-level knowledge distillation on CPU hardware can achieve production-grade natural language command parsing (**99.0% JSON validity** and **46.0% full-frame accuracy**). Deploying the model inside an optimized container (<1.5 GB) on a local Kubernetes cluster with Traefik ingress, HPA autoscaling, stateless NACLs, and automated CI/CD quality gates provides a highly resilient, cost-effective (■0.26 / 1,000 requests), and reproducible cloud architecture.

References: (1) Kim & Rush, *Sequence-Level Knowledge Distillation*, EMNLP 2016. (2) Bastianelli et al., *SLURP Dataset*, EMNLP 2020. (3) Biderman et al., *Pythia Suite*, ICML 2023. (4) Google SRE Book, *Service Level Objectives*.